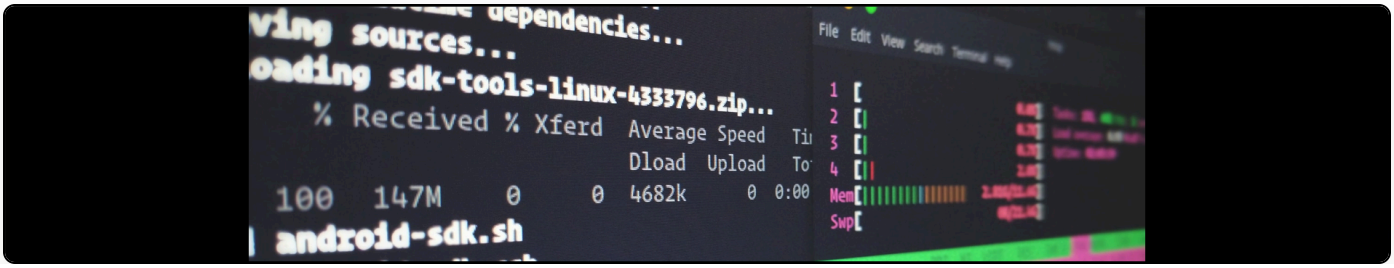
C# MemoryManagement Class

Cite as: Martin Paul Eve, "C# MemoryManagement Class", *eve.gd: Martin Paul Eve*, July 06, 2008, https://doi.org/10.59348/9m7s6-f3k18.



Well, first off, this is the first post using the new blogging solution! Let's hope it works!

I'm presenting here a low level memory management class I wrote for C# that allows you to pass an IntPtr and then manipulate the bytes inside as indexers of an array (of the MemoryManager object).

```
using System;
using System.Collections.Generic;
using System.Text;
using System.Runtime.InteropServices;

namespace Switch
{

    public class MemoryManager
    {
        public IntPtr mmPtr = IntPtr.Zero;

        public byte this[int index]
        {
            get
            {
                return Marshal.ReadByte(mmPtr, index);
            }
            set
            {
                Marshal.WriteByte(mmPtr, index, value);
            }
        }

        public byte[] this[int startindex, int endindex]
        {
            get
            {
                byte[] ret = new byte[endindex - startindex];

                for (int y = startindex; y < endindex; y++)
                {
                    ret[y - startindex] = Marshal.ReadByte(mmPtr, y);
                }

                return ret;
            }
            set
            {
                for (int y = startindex; y < endindex; y++)
                {
                    Marshal.WriteByte(mmPtr, y, value[y - startindex]);
                }
            }
        }

        public void SubtractInt(int index, uint value)
```

```
        {
            Byte[] bytes = BitConverter.GetBytes(((uint)(Marshal.ReadInt32(mmPtr,
index))) - value);

            Marshal.WriteByte(mmPtr, index, bytes[0]);
            Marshal.WriteByte(mmPtr, index + 1, bytes[1]);
            Marshal.WriteByte(mmPtr, index + 2, bytes[2]);
            Marshal.WriteByte(mmPtr, index + 3, bytes[3]);
        }

        public int WriteStringToMemory(string s, int offset)
        {
            int writtenCount = 0;

            for (int charByteCount = 0; charByteCount < s.Length; charByteCount++)
            {
                Byte[] charConverted = BitConverter.GetBytes(s[charByteCount]);
                Marshal.WriteByte(mmPtr, charByteCount + offset, charConverted[0]);

                writtenCount++;
            }

            return writtenCount;
        }

        public MemoryManager(IntPtr start)
        {
            mmPtr = start;
        }

        public static string plainText(IntPtr buffer, bool changePlain)
        {
            string ret = string.Empty;

            for (int y = 0; y < 64; y++)
            {
                Byte b = Marshal.ReadByte(buffer, (y == 0 && changePlain ? 60 : y));

                if (b == 0x80) return ret;

                ret += (char)b;
            }

            return ret;
        }


        public void WriteUintToMemory(int offset, uint data)
```

```
        {
            writeOrd32Chunk(offset, BitConverter.GetBytes(data));
        }

        public void WriteByteToMemory(int offset, byte data)
        {
            Marshal.WriteByte(mmPtr, offset, data);
        }
    }
}
```

As an example of how to use this class (although perhaps not *why* you might want to!), consider the following:

```
        //Setup buffer
        bufferPtr = Marshal.AllocHGlobal(64);
        MemoryManager mmBuffer = new MemoryManager(bufferPtr);

        Console.WriteLine("bufferPtr begins at {0} and occupies {1} bytes.",
bufferPtr.ToString(), 64);

        //Write zeroes from 0->56 (14*4)
        mmBuffer.WriteZeros(0, 14);
        Console.WriteLine("buffer[0->56] written with zeroes.");

        //Write the start value
        mmBuffer.WriteStringToMemory("astring", 0);
        Console.WriteLine("String written to buffer[0].");

        //Write the padding marker
        mmBuffer[searchLen] = 0x80;
        Console.WriteLine("Padding marker (0x80) written to buffer[" +
searchLen.ToString() + "].");

        //Write the tail at position 56
        mmBuffer.WriteUintToMemory(56, (uint)tailStringPtr);
        Console.WriteLine("tailStringPtr written to buffer[56].");

        //Write the working area to position 60->63
        mmBuffer[60, 63] = mmBuffer[0, 3];
```

This class is primarily useful when you need to pass a memory structure via a pInvoke and want to easily manipulate the data.